

Notes

These notes should serve as aid for the implementation, and will eventually be integrated into the report;

1 GMRES

We want to find:

$$\min_{x \in K_n(A, y)} \|Ax - y\|$$

1.1 Background: Krylov Spaces and Arnoldi

Given a non-necessarily-symmetric matrix (although in our case it is) $A \in \mathbb{R}^{m \times m}$, a vector $y \in \mathbb{R}^m$, and a natural number $n \leq m$:

$$K_n(A, y) = \text{span}(y, Ay, A^2y, \dots, A^{n-1}y)$$

We can obtain a decent basis for this vector space by running the Arnoldi Algorithm:

$$\text{Arnoldi}(A, y, n) = Q_n, \underline{H}_n$$

Where:

- $Q_n \in \mathbb{R}^{n \times m}$ is the matrix $(q_1 \mid \dots \mid q_n)$ of vectors in the basis
- $\underline{H}_n \in \mathbb{R}^{n+1 \times n}$ is a matrix such that:

1.

$$\underline{H}_n = \begin{bmatrix} H_n \\ \dots \end{bmatrix}$$

where $H_n \in \mathbb{R}^{n \times n}$ is s.t. $H_n = Q_n^T A Q_n$, and can therefore be seen as A in the Krylov subspace.

2.

$$A Q_n = Q_{n+1} \underline{H}_n \tag{1}$$

Remark 1 The first vector of the basis generated by $\text{Arnoldi}(A, y, n)$ is $\frac{y}{\|y\|}$.

1.2 Simplifying the minimum problem

We simplify the minimum problem as follows:

$$\begin{aligned} \min_{x \in K_n(A, y)} \|Ax - y\| &= \min_{z \in \mathbb{R}} \|A Q_n z - y\| && (q_i \text{ s are basis of } K_n(A, y)) \\ &= \min_{z \in \mathbb{R}} \|Q_{n+1} \underline{H}_n z - y\| && (\text{by (1)}) \\ &= \min_{z \in \mathbb{R}} \|Q_{n+1}^T (Q_{n+1} \underline{H}_n z - y)\| && (Q_{n+1}^T \text{ orthogonal}) \\ &= \min_{z \in \mathbb{R}} \|\underline{H}_n z - Q_{n+1}^T y\| && (Q_{n+1} \text{ orthogonal}) \\ &= \min_{z \in \mathbb{R}} \|\underline{H}_n z - Q_{n+1}^T q_1 \|y\|\| && (\text{by Remark 1}) \\ &= \min_{z \in \mathbb{R}} \|\underline{H}_n z - e_1 \|y\|\| && (Q_{n+1}^T \text{ orthogonal}) \end{aligned}$$

1.3 Lanczos

Since the input matrix A is symmetric, we can use the Lanczos Algorithm to iteratively construct the orthonormal basis for the Krylov space. In particular, the H_n matrix resulting from such algorithm will be symmetric and tridiagonal:

$$H_n = \begin{bmatrix} \alpha_1 & \beta_1 & & & & \\ \beta_1 & \alpha_2 & \beta_2 & & & \\ & \beta_2 & \alpha_3 & \ddots & & \\ & & \ddots & \ddots & \beta_{m-2} & \\ & & & \beta_{m-2} & \alpha_{m-1} & \beta_{m-1} \\ & & & & \beta_{m-1} & \alpha_m \end{bmatrix}$$

This allows us to perform the computations without storing the whole matrix, but only the diagonal and subdiagonal. The algorithm is as follows: **TODO: Write only pseudocode**

```

1  function Lanczos(A::SparseMatrixCSC, y::Vector, n::Int)
2
3      alpha = zeros(1,n)      # diagonal of H
4      beta = zeros(1,n)      # subdiagonal of H (= superdiagonal of H because symmetric)
5
6      L = zeros(size(A)[1], n)  # Lanczos Basis
7
8      # base case
9
10     q = y / norm(y)
11     L[:, 1] = q
12     w = A * q
13     alpha[1] = w' * q
14     w = w - alpha[1] * q
15     beta[1] = norm(w)
16     # inductive case
17
18     for j in 2:n
19         if beta[j-1] < 1e-13
20             println("Breakdown");
21             return
22         end
23         q = w / beta[j-1]
24         L[:, j] = q
25
26         w = A * q
27         alpha[j] = w' * q
28         w = w - alpha[j] * q - beta[j-1] * L[:, j-1]
29         println("w")
30         display(w)
31         beta[j] = norm(w)
32
33     end
34
35     return (L, alpha, beta)
36
37 end

```

1.4 Lanczos + QR

A trivial way to implement GMRES would be:

- Use Lanczos to obtain a basis for the Krylov space
- Compute the QR factorization of $\underline{H}_n$
- Use the factorization to easily solve the linear system arising from the minimum problem.

The requested optimization asks to compute the QR factorization during the Lanczos iteration.

1.4.1 Extending the QR decomposition

Assume we have $H_{n-1} = Q_{old}R_{old}$. H_n will be computed using a Lanczos iteration, obtaining:

$$H_n = \left[\begin{array}{c|c} & \begin{matrix} 0 \\ \vdots \\ 0 \end{matrix} \\ \hline H_{n-1} & \begin{matrix} \beta_{n-1} \\ \alpha_n \end{matrix} \\ \hline \begin{matrix} 0 & \dots & 0 \end{matrix} & \beta_n \end{array} \right]$$

We want to find two matrices Q , R such that:

- $Q \in \mathbb{R}^{(n+1) \times (n+1)}$ is orthogonal
- $R \in \mathbb{R}^{(n+1) \times n}$ is upper triangular
- $H_n = QR$, i.e. $Q^T H_n = R$

First off, to obtain a Q matrix of the requested shape that is orthogonal, we could consider:

$$Q'_{old} = \left[\begin{array}{c|c} & \begin{matrix} 0 \\ \vdots \\ 0 \end{matrix} \\ \hline Q_{old} & \begin{matrix} \beta_{n-1} \\ \alpha_n \end{matrix} \\ \hline \begin{matrix} 0 & \dots & 0 \end{matrix} & \beta_n \end{array} \right]$$

This takes us close to the solution, but not quite there; in fact the product $Q'^T_{old} H_n$ is *almost* triangular:

$$Q'^T_{old} H_n = \left[\begin{array}{c|c} & \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \beta_{n-1} \\ \alpha_n \end{bmatrix} \\ \hline R_{old} & \begin{bmatrix} \beta_{n-1} \\ \alpha_n \end{bmatrix} \\ \hline \begin{matrix} 0 & \dots & 0 \end{matrix} & \beta_n \end{array} \right]$$

Recall that R_{old} is an $n \times (n-1)$ upper triangular matrix, hence it has a row of zeros at the bottom. To make this matrix triangular, we should have a zero in place of the β_n . We can get rid of such zero by performing a Householder reflection on the bottom two elements of the final column. Let these be:

$$x = \begin{bmatrix} a \\ \beta_n \end{bmatrix} \quad \mapsto \quad y = \begin{bmatrix} \|x\| \\ 0 \end{bmatrix}$$

As seen in class, the transformation we need is a reflection w.r.t $v = x - y$:

$$H_{refl} = I - \frac{2vv^T}{\|v\|^2}$$

In order for the reflection to only act on the two bottom elements of the final row, we extend it as follows, obtaining another orthogonal transformation:

$$O = \left[\begin{array}{c|cc} & 0 & 0 \\ & \vdots & \vdots \\ & 0 & 0 \\ \hline 0 \dots 0 & & \\ 0 \dots 0 & H_{refl} & \end{array} \right]$$

The matrix we obtain as $OQ'_{old}H_n$ is triangular.

Constructing the R Matrix

- Get old R ;
- Compute:

$$Q'_{old}[0, \dots, 0, \beta_{n-1}, \alpha_n]^T = ([0, \dots, 0, \beta_{n-1}, \alpha_n]Q'_{old})^T$$

Let $q_1 \dots q_n$ be the columns of Q'_{old} . (TODO rename Lanczos basis vector to b_i ?) The result is:

$$\sum_i \beta_{n-1} q_{i,n-1} + \alpha_n q_{i,n}$$

This can be rewritten, letting $r_1 \dots r_n$ be the rows of Q'_{old} , as:

$$\beta_{n-1} r_{n-1} + \alpha_n r_n$$

- Perform the O transformation to obtain R .

Constructing the Q matrix

- Get old Q ;
- Construct Q'_{old} by adding the extra row and column;
- Compute $Q = (OQ'_{old})^T$.

Altenatively we can calculate the new Q as follows: